

Graph

Q.N0:1

Question description

Teddy is visiting the country wonderland. wonderland has n cities and m bi-directional roads. There are k types of tokens. Token i costs c_i . The costs of the tokens are such that for all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$. For each road, you need to have a particular set of tokens, if you want to travel it. Note that you don't have to give the tokens, you just need to show them. Thus, one token can be used at any number of roads, where it is required. Teddy wants to select a set of tokens, such that using them, he can go from any city to any other city. You have to help him minimize the total cost of tokens he buys.

Constraints

$$1 \leq n \leq 10^5$$

$$1 \leq m \leq 10^5$$

No road connects a city to the same city. However, there can be multiple roads between two cities.

$$1 \leq c_i \leq 10^{18}$$

For all $2 \leq i \leq k$, $c_i \geq 2c_{i-1}$

Input:

- The first line contains three space separated integers, n m and k .
- The second line contains k space separated integers, where the i^{th} integer denotes the price of i^{th} token, i.e. c_i .
- i^{th} of the next m lines contains three integers u_i, v_i, l_i , where l_i is the number of tokens required by the i^{th} road, followed by l_i indices denoting the tokens required. This road connects cities u_i and v_i .

Output:

- Print one integer containing the minimum cost of tokens Teddy has to buy, such that he can travel from any city to any other city. If it is impossible to choose such a set of tokens, print -1 .

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
```

EXPECTED OUTPUT

8

Test Case 2

INPUT (STDIN)

```
3 3 4
11 21 15 10
1 21 2 11 2
1 13 1 3
2 31 1 4
```

EXPECTED OUTPUT

57

Code:

```
#include <stdio.h>
#include <stdlib.h>

#define MAXN 100005
#define MAXM 100005
#define MAXK 65
```

```
int parent[MAXN];
int size[MAXN];

int u[MAXM], v[MAXM], l[MAXM];
int req[MAXM][MAXK];

long long cost[MAXK];

int find(int x) {
    if (parent[x] == x)
        return x;

    return parent[x] = find(parent[x]);
}

void unite(int a, int b) {
    a = find(a);
    b = find(b);

    if (a == b)
        return;

    if (size[a] < size[b]) {
        int temp = a;
        a = b;
        b = temp;
    }
    parent[b] = a;
    size[a] += size[b];
}

int connected(int n, int m, int k, int available[]) {
    for (int i = 1; i <= n; i++) {
        parent[i] = i;
        size[i] = 1;
    }
    for (int i = 0; i < m; i++) {
        int ok = 1;
        for (int j = 0; j < l[i]; j++) {
            if (!available[req[i][j]]) {
                ok = 0;
                break;
            }
        }
        if (ok)
            unite(u[i], v[i]);
    }
    int root = find(1);

    for (int i = 2; i <= n; i++) {
        if (find(i) != root)
            return 0;
    }
    return 1;
}
```

```
}
int main() {
int n, m, k;
scanf("%d %d %d", &n, &m, &k);

for (int i = 1; i <= k; i++)
scanf("%lld", &cost[i]);

for (int i = 0; i < m; i++) {
scanf("%d %d %d", &u[i], &v[i], &l[i]);

for (int j = 0; j < l[i]; j++)
scanf("%d", &req[i][j]);
}
int available[MAXK];
for (int i = 1; i <= k; i++)
available[i] = 1;
if (!connected(n, m, k, available)) {
printf("-1\n");
return 0;
}
for (int i = k; i >= 1; i--) {
available[i] = 0;
if (!connected(n, m, k, available))
available[i] = 1;
}
long long answer = 0;
for (int i = 1; i <= k; i++) {
if (available[i])
answer += cost[i];
}
printf("%lld\n", answer);

return 0;
}
```

Output:

Test_case:1

```
isc 7graph/1
3 3 4
1 2 5 10
1 2 2 1 2
1 3 1 3
2 3 1 4
Output:
8
aadityajaiswal@Aadit
```

Test_case:2

```
aadityajaisw  
tise /graph/  
3 3 4  
11 21 15 10  
1 2 1 2  
1 3 1 3  
2 3 1 4  
Output:  
57
```